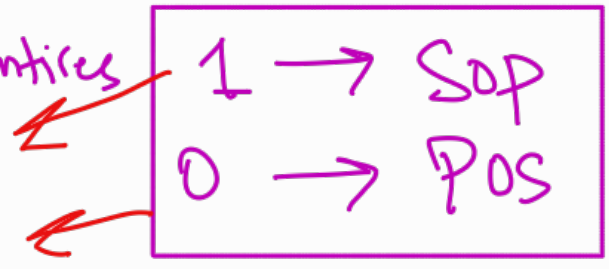
K-maps: A graphical method for representation and minimization of Boolean functions.

↓

pictorial represent of Truth-table

K-maps is an array's of cells. The cells are arranged in a way so that simplification of given expression is just a matter of properly grouping the cells.

K-maps with entries



no. of cells 2^n ← for n variable function.

One cell represent one term 

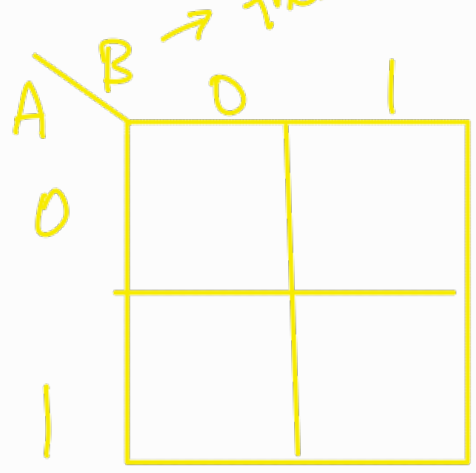
no. of cells of K map is equal to no. of input

For 4 variables $2^4 = 16$ cells

2-variables K-map

A	B
0	0
0	1
1	0
1	1

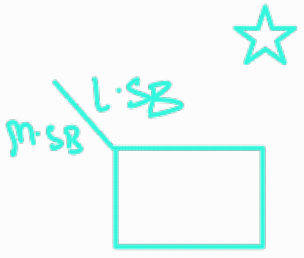
T.T



k-map

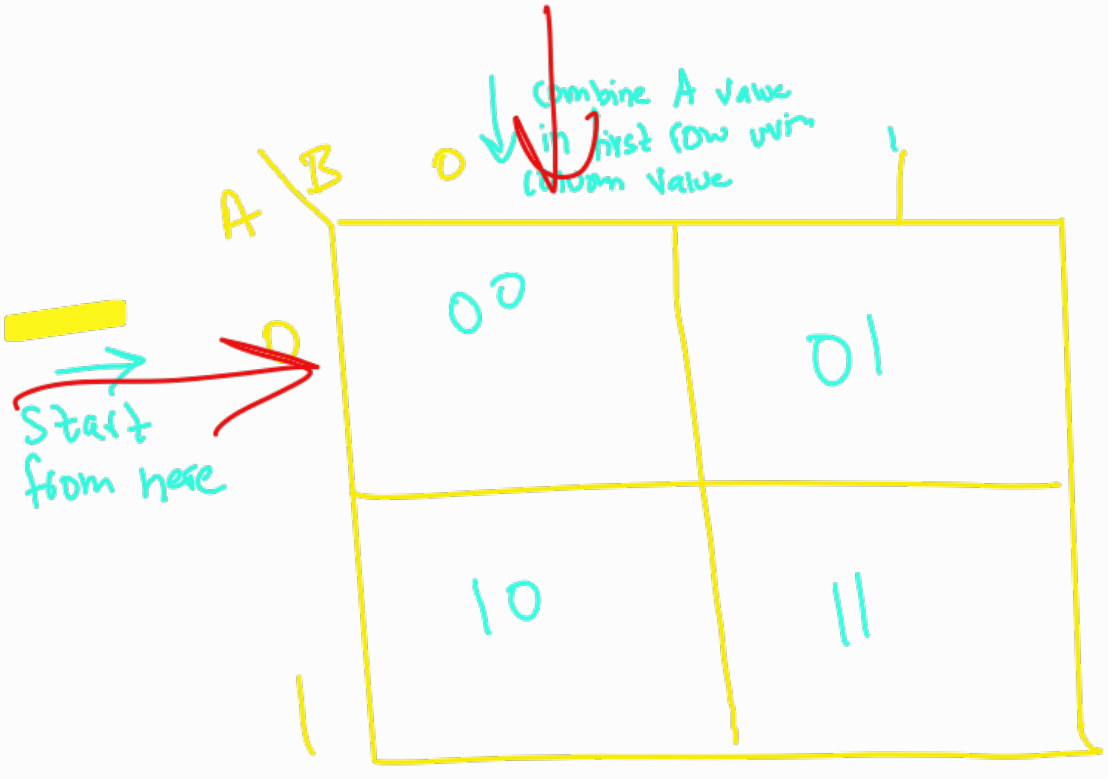
→ these are input variables

4-cells



Possible values of A. 1 variable

2¹ → 2 Same with B.



Rewriting the k-map in Sop form

0 → complement 1 → uncomplement

A \ B	$\bar{B}$	B
$\bar{A}$	$\bar{A}\cdot\bar{B}$	$\bar{A}\cdot B$
A	$A\cdot\bar{B}$	$A\cdot B$

note:- By default we fill up the cells of k-map with min-terms (1's. If not mentioned in question.

A	B	F
0	0	1 ←
0	1	1 ←
1	0	0
1	1	1 ←

A \ B	0	1
0	1	1
1		1

Inputs

outputs.

A	B	F
0	0	1
0	1	1
1	0	0
1	1	1

writing SOP equation from T.T

$\bar{A}\cdot\bar{B} + \bar{A}\cdot B + AB$

$\bar{A}(\bar{B}+B) + AB$

$\bar{A} + AB$ using distributive law

$(\bar{A}+A)(\bar{A}+B)$

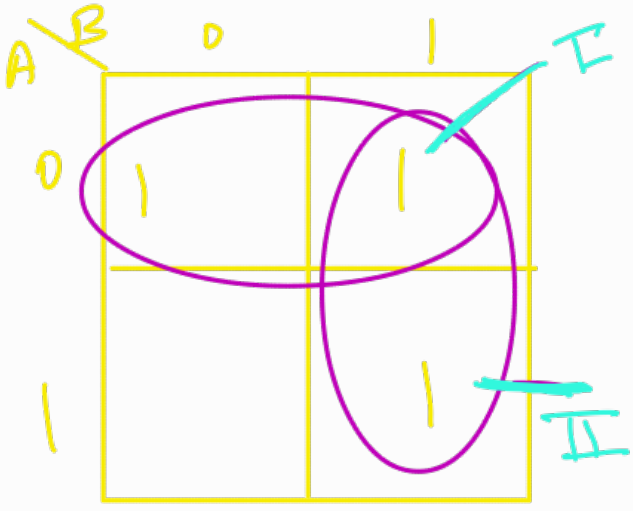
$\boxed{\bar{A}+B}$ final

But using k-maps we can directly reach to this eq

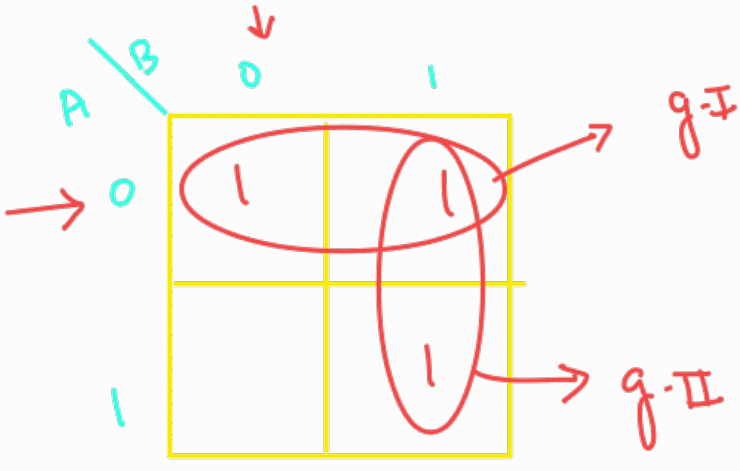
Rules of Making groups of 1.

→ We cannot make groups diagonally. We can make groups with neighbaring / adjacent cells.

→ Every 1 in k-map must be mapped in group.



For writing expression
Start from Row and then move to columns.



Discard the variable whose Input has Change here B in first column changes (0→1)

g.I

A = 0

B = 0 then

B = 1

Discard B

Ans = $\overline{A}$

---

g.II

A = 0 then

A = 1

B = 1

Ans = B

Write in Sop form

$\overline{A} + B$

$\overline{A}$ ↓ grp I

B ↓ grp II

← (look at Input values) only not in cell values

Discard A as its value changes.

(3 variable K-map) $2^3 = 8$ cells

MSB ↑

A	B	C → LSB
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

→

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

writing in SoP and marking decimal equivalent at top right corner.

A \ BC	00	01	11	10
0	$\bar{A}\cdot\bar{B}\cdot\bar{C}$ 0	$\bar{A}\cdot\bar{B}\cdot C$ 1	$\bar{A}\cdot B\cdot C$ 3	$\bar{A}\cdot B\cdot\bar{C}$ 2
1	$A\cdot\bar{B}\cdot\bar{C}$ 4	$A\cdot\bar{B}\cdot C$ 5	$A\cdot B\cdot C$ 7	$A\cdot B\cdot\bar{C}$ 6

Another way to write 3-Kmap $2^3 = 8$ cells

AB \ C	0	1
00	000	001
01	010	011
11	110	111
10	100	101

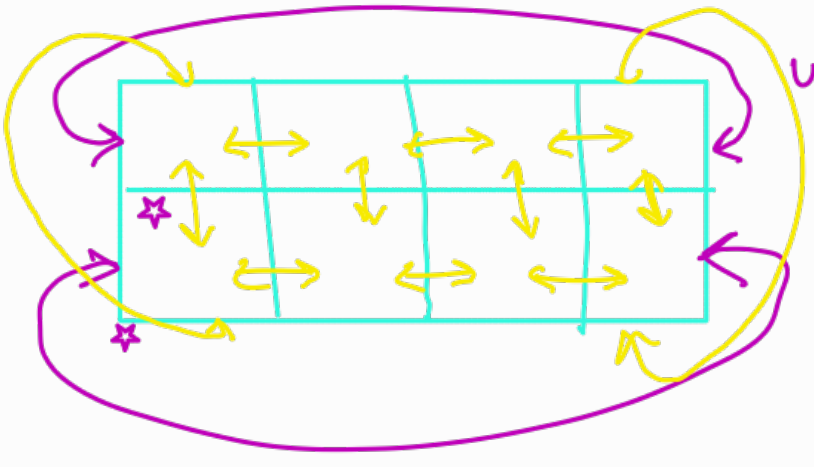
(4 variables k-map)

A → MSB
D → LSB

$2^4 = 16$ cells.

AB \ CD	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

Adjancency in k-maps. meaning ability to make groups if 1 is present



wrap around adjancency

wrap around adjancency.

To confirm adjacent cells.

→ cells that differ by only one variable are adjacent | two variables must have same values.

→ cells that differ by more than one variable are not adjacent

e.g

A	B	C
0	1	0
0	0	0

— adjacent

A	B	C
0	1	0
0	0	1

not adjacent

Non-adjacent cannot make groups.

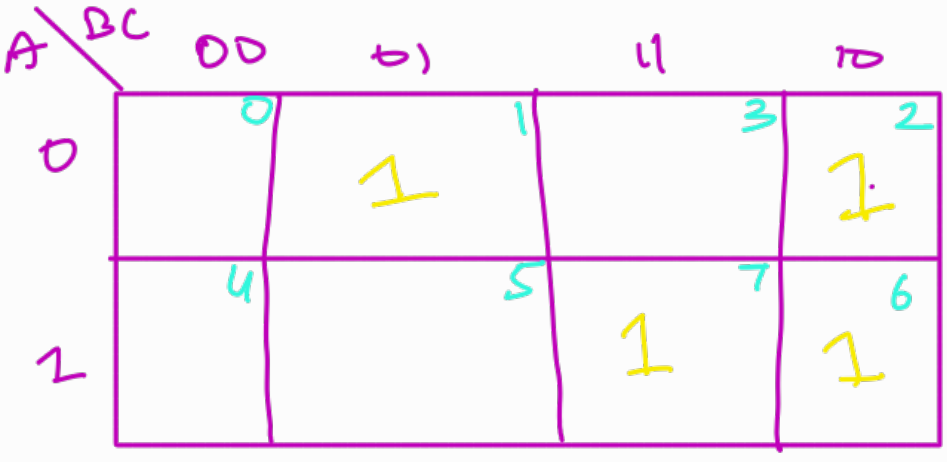
Mapping SSOP on K-map.

1) Determine binary values of each product term in the expression.

2) Place a 1 on K-map in the cells having same value as product term

Map on K-map. $\bar{A}\cdot\bar{B}\cdot C + \bar{A}\cdot B\cdot\bar{C} + A\cdot B\cdot\bar{C} + A\cdot B\cdot C$

no. of cells = $2^3 = 8$



Writing this in Binary equavalent.

001 = $\bar{A}\cdot\bar{B}\cdot C$

010 = $\bar{A}\cdot B\cdot\bar{C}$

110 = $A\cdot B\cdot\bar{C}$

111 = $A\cdot B\cdot C$

→ Place 1 in K-map according to the given decimal value.

Mapping SOP on K-map

note: we can plot only standard values in K-map. / SSOP / SPOS

hence we need to convert SOP → SSOP by completing the missing variables.

Suppose we have 3 variable expression and one of the terms is not filled.

..... + A·C + ... in SOP

this means B is missing

A and C hold value of 1 as they are non-complement

A	B	C
1	?	1

→

A	B	C
1	0	1
1	1	1

B can either be complemented / un-complemented

A C are now fixed

Another e.g we have 3 variable K-map. A and C are missing. $\overline{B}$ value is fixed 0

..... + $\overline{B}$ + ...

A	B	C
1	0	1
0	0	1
1	0	0
0	0	0

As two variables are missing

2^2 = 4 combinations missing

$$\overline{A} + A\cdot\overline{B} + A\cdot B\cdot\overline{C}$$

completing SSOP for $\overline{A}$

Two variables missing B and C hence $2^2 = 4$ combinations missing

A	B	C
0	1	1
0	0	1
0	1	0
0	0	0

→ map all of these on k-map

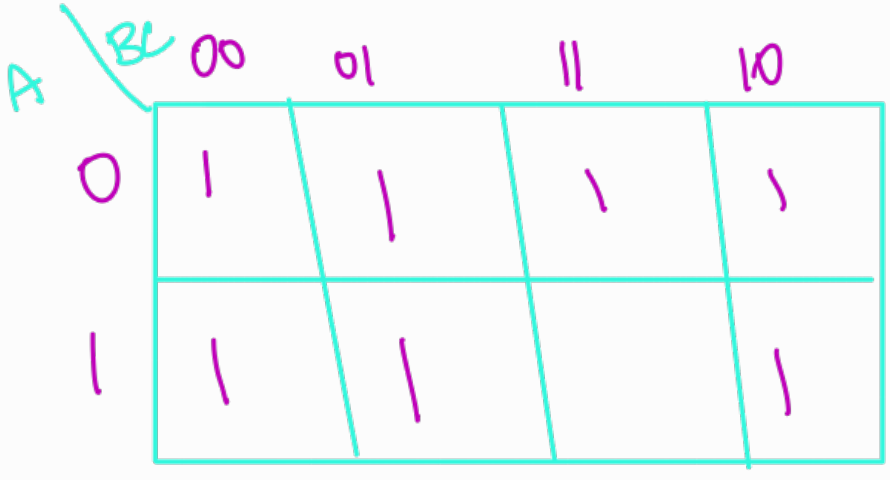
completing SSOP for $A\cdot\overline{B}\cdot$ Missing is C hence $2^1 = 2$ combinations missing

A	B	C
1	0	0
1	0	1

Map SOP on k-map

2^3 = cells

$$\overline{A} + A\cdot\overline{B} + A\cdot B\cdot\overline{C}$$



Map Sop on k-map

$\overline{B} \cdot \overline{C} + A \cdot \overline{B} + A \cdot B \cdot \overline{C} + A \cdot \overline{B} \cdot C \cdot \overline{D} + \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot D +$

$A \cdot \overline{B} \cdot CD$

Missing A and D for $\overline{B} \cdot \overline{C}$

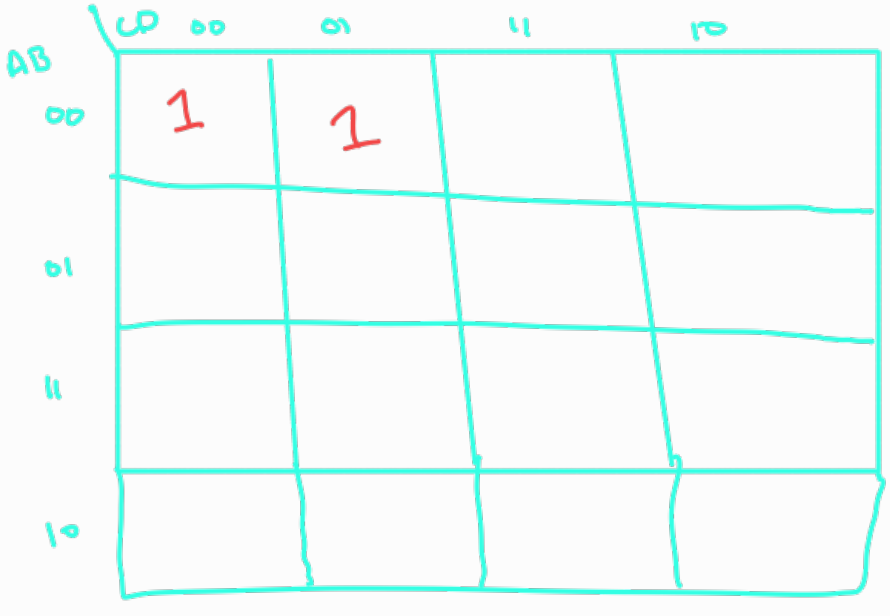
$2^2 = 4$

A	B	C	D
1	0	0	1
0	0	0	0
1	0	0	0
0	0	0	1

missing C and D for $A \cdot \overline{B}$ $2^2 = 4$

A	B	C	D
1	0	0	0
1	0	1	0
1	0	0	1
1	0	1	1

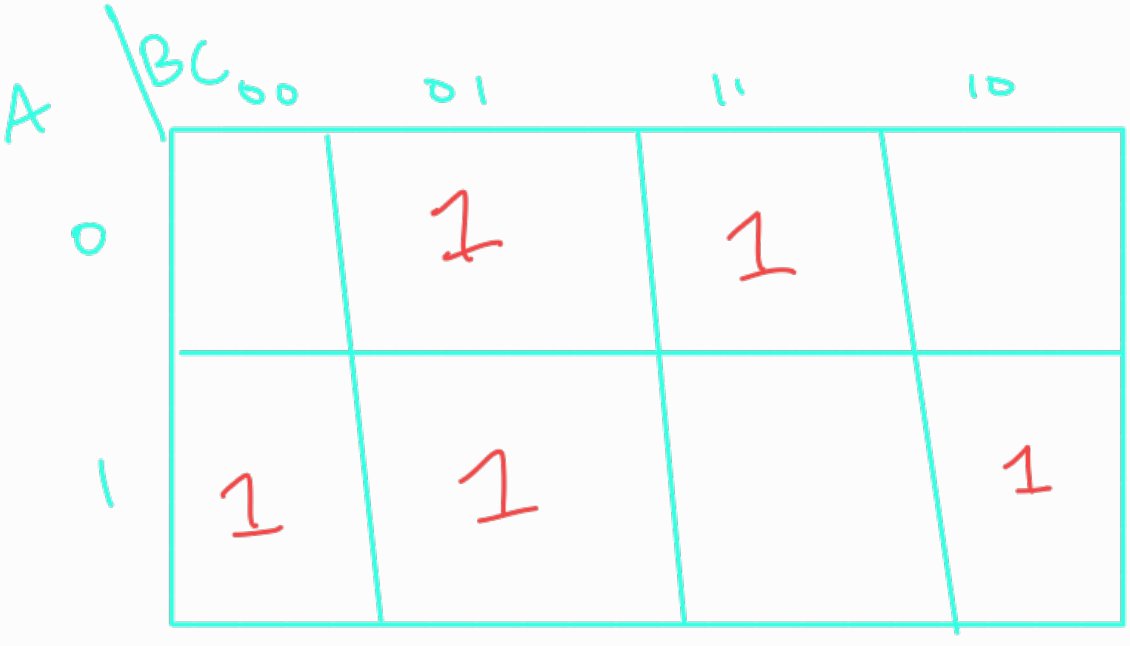
$2^4 = 16$ cells k-map



if Binary Value is repeated Just mark it once on k-maps

Map $F = \Sigma m(1, 3, 4, 5, 6)$ on k-map

3 variable k-map as largest decimal value is 6



Write the decimal equivalent then map

1 → 001

3 → 011

4 → 100

5 → 101

6 → 110

In Sop your are required to use 1.

In pos your are requires to use 0.

How to make groups on k-map of 1's

Points to remember for making groups of 1.

1) A group must contain 1, 2, 4, 8, 16 cells ($2^0 = 1$, $2^1 = 2$, $2^2 = 4$) ← no. of ones (Powers of 2 Rule)

2)

A \ BC	00	01	11	10
0		1	1	1
1	1		1	1

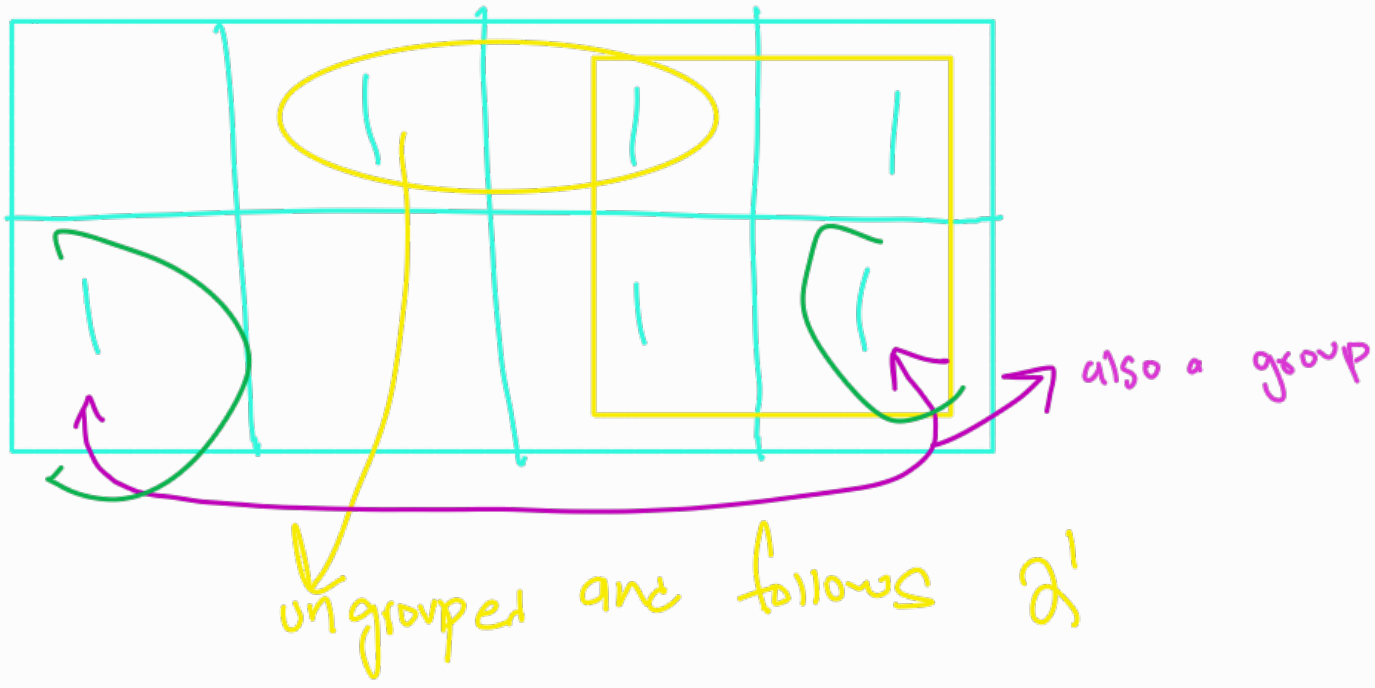
2^0 group

Always include the largest possible no. of 1s in a group. while following adjency group.

3) each 1 on map must be included in at least one group.

Make sure grouping of 1 is largest

The 1 already included in a group can also be included in an another group as long as overlapping group contains ungrouped 1's.

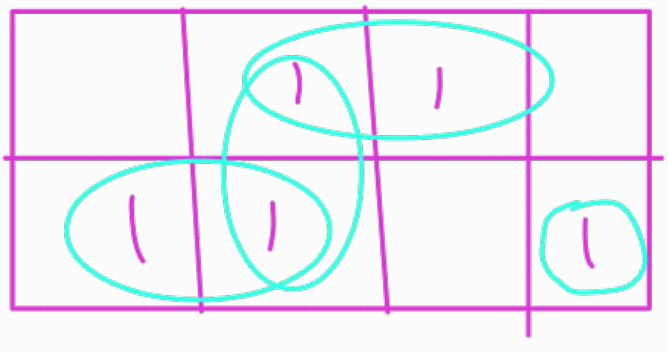


e.g of grouping

A \ BC	00	01	11	10
0	1	1	1	1
1	1	1		

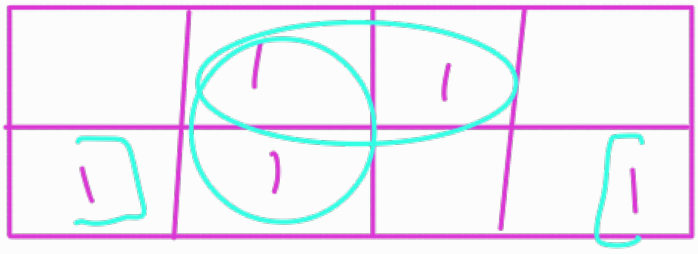
Each group of 1's should be as large as possible as there must be few groups

e.g



×

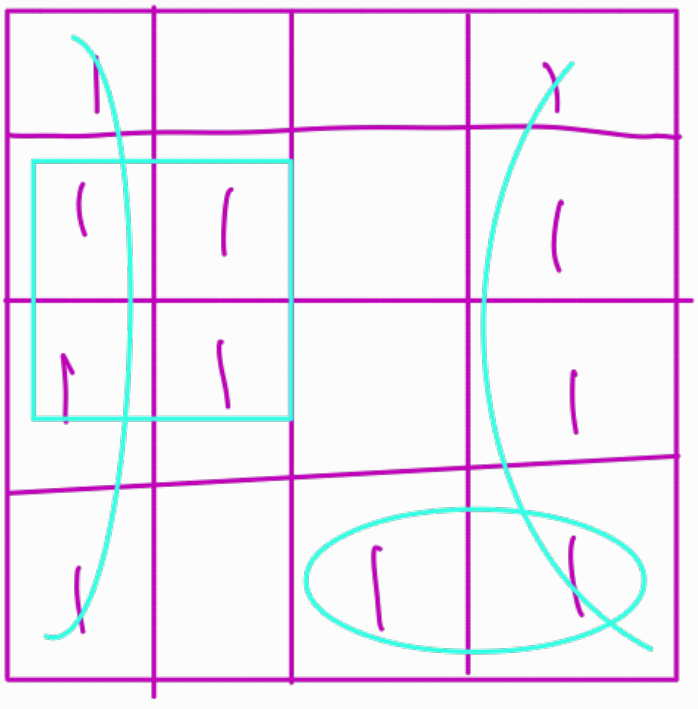
as over here we have 4 groups.

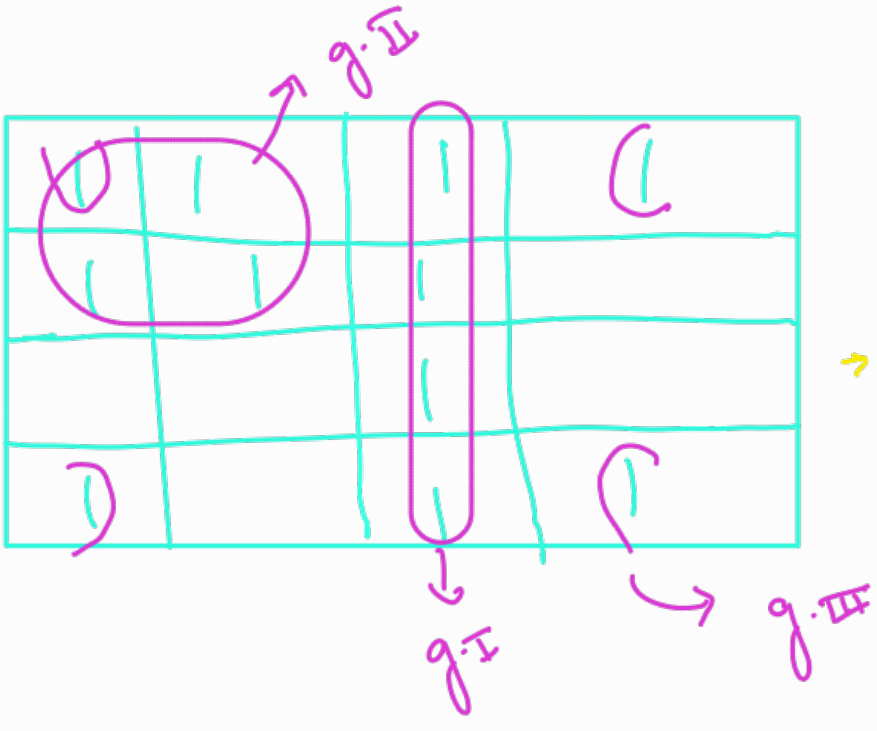


✓

But here we have only 3 groups. containing max no of 1.

Mapping examples

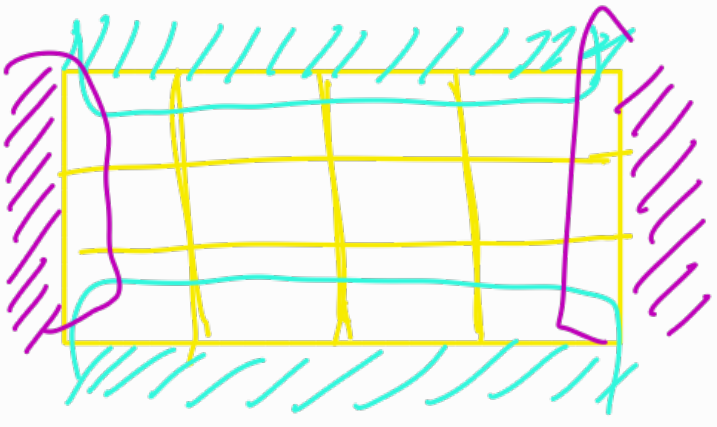


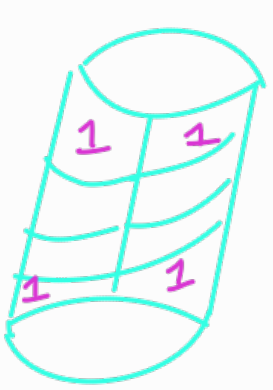


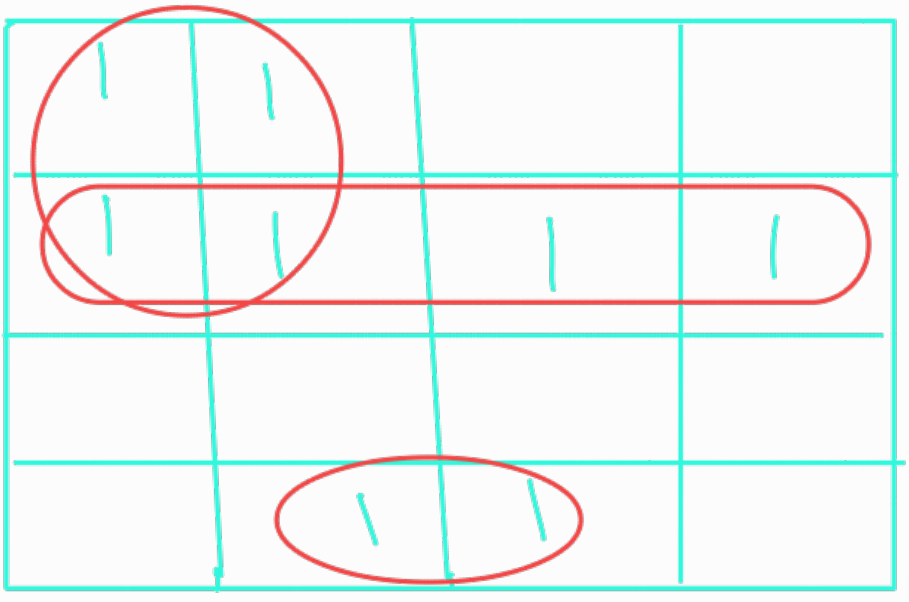
total no. of groups
= 3

→ four colours are said to be adjacent of each other

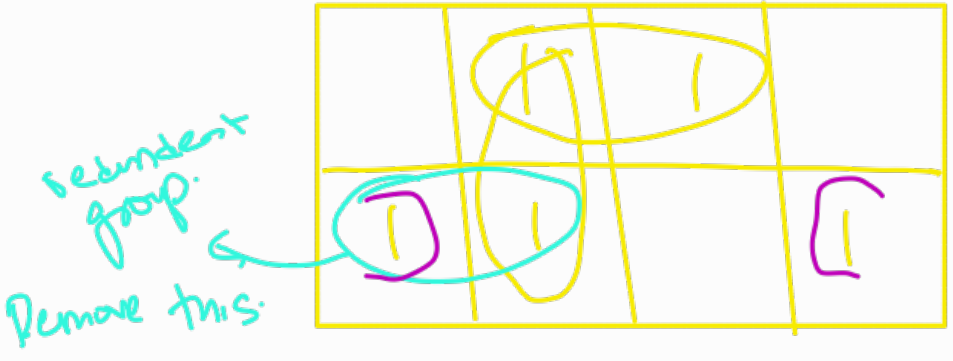
adjacent







note:: Remove the redundant group.



Determining the min SOP expression from K-map.

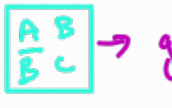
After mapping all the K-maps and grouping all the 1's. Write the resultant minized SOP from K-map (Steps of writing resultant expression)

→ group the cells that have 1s. Each group creates one product term.

↓

consist of variables that are either complemented/ uncomplemented but not Both.

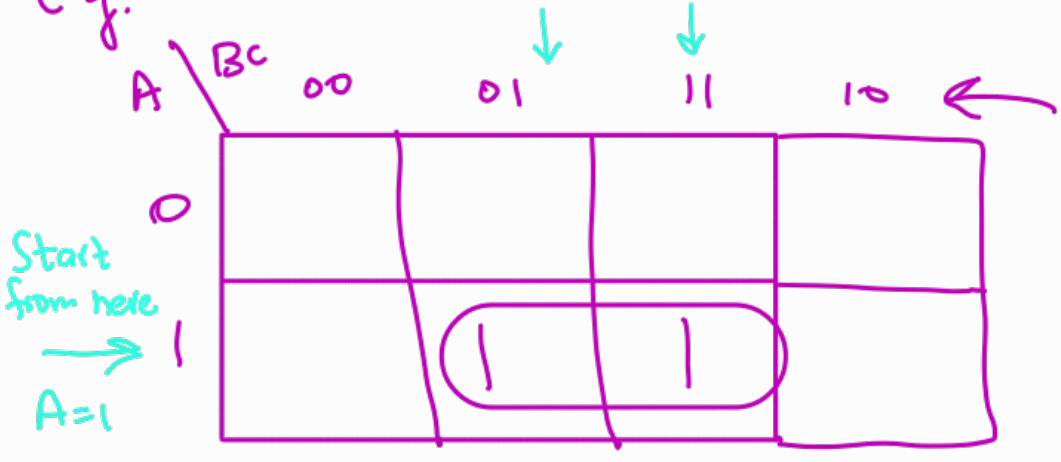
Make Sure to eliminate variables that exist in Both forms a single group. ($\bar{A}$ and A)



→ group eliminate $\frac{B}{\bar{B}}$ and

→ Lastly Sum of all minmun product terms.

e.g.



Determine variables by looking at the inputs

Product term = $\bar{A}C$

B (column)

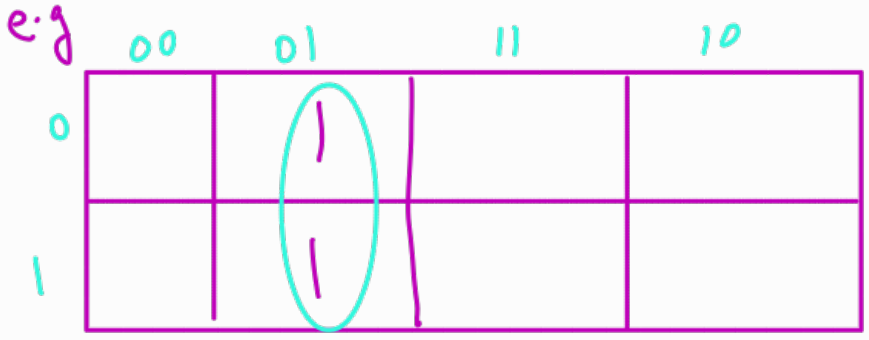
0 → C

1 → C=1.

1

Discard as its value changes.

e.g

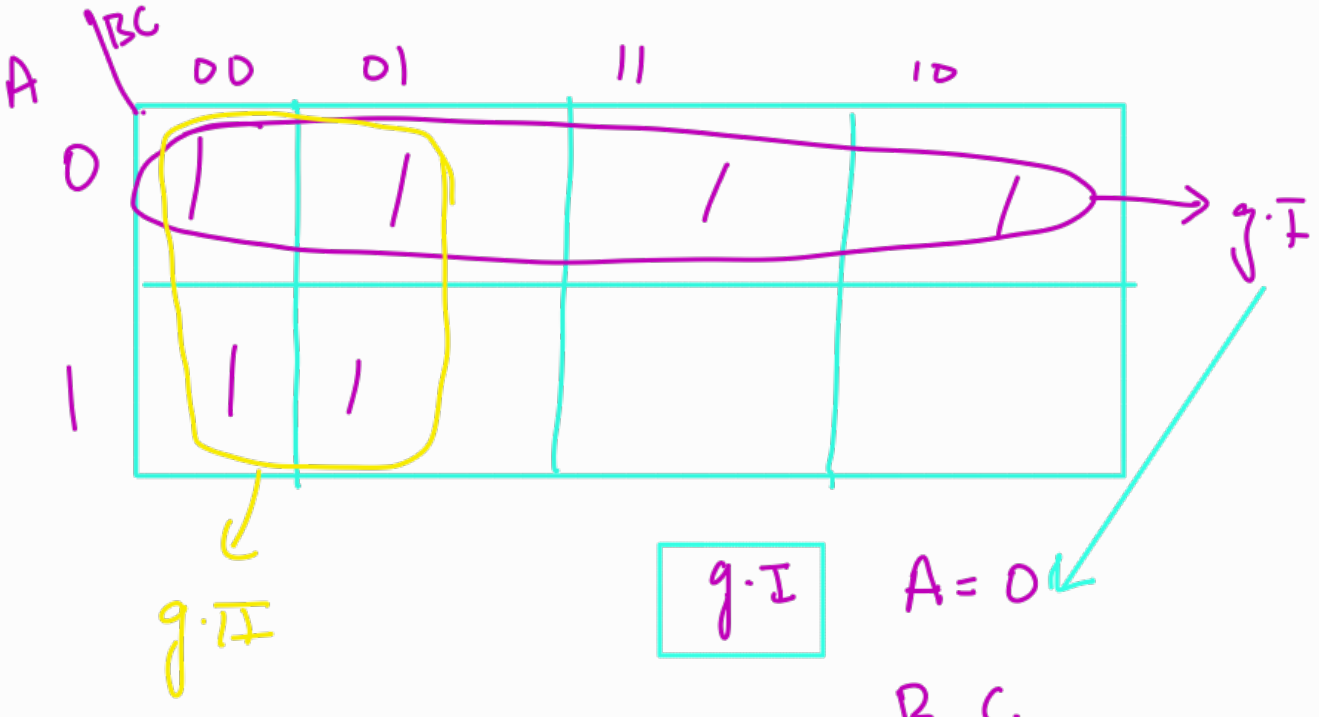


A=0
A=1

Discard A as its value contradicts.

B=0 C=1

final = $\overline{B} \cdot C$



g·I

A=0

B	C	
0	0	→ discard Both.
0	1	
1	1	
1	0	

write the expressions for each group seperately one at a time. and start from Row to column.

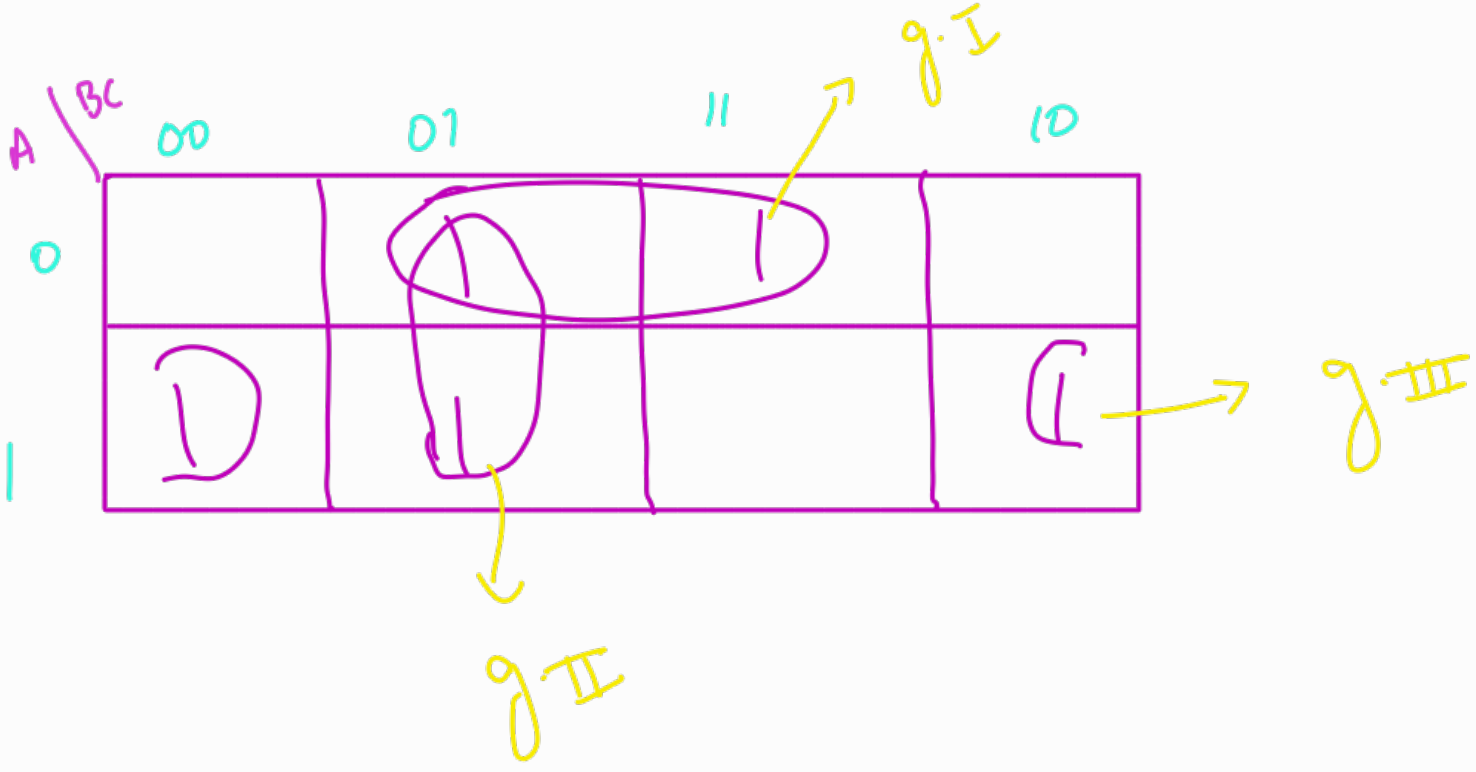
for g.II

A	B	C
0	0	0
0	0	1
1		

(X) (X) → $\overline{B}$

final = $\boxed{\overline{A} + \overline{B}}$

Ans



g.I

A = 0

B	C
0	1
1	1

(X)

final $\overline{A} \cdot C$

g.II

A = 0 then A = 1

(X)

B = 0 C = 1

final = $\overline{B} \cdot C$

g·III

A = 1 B C final = $A \cdot \bar{C}$

B C

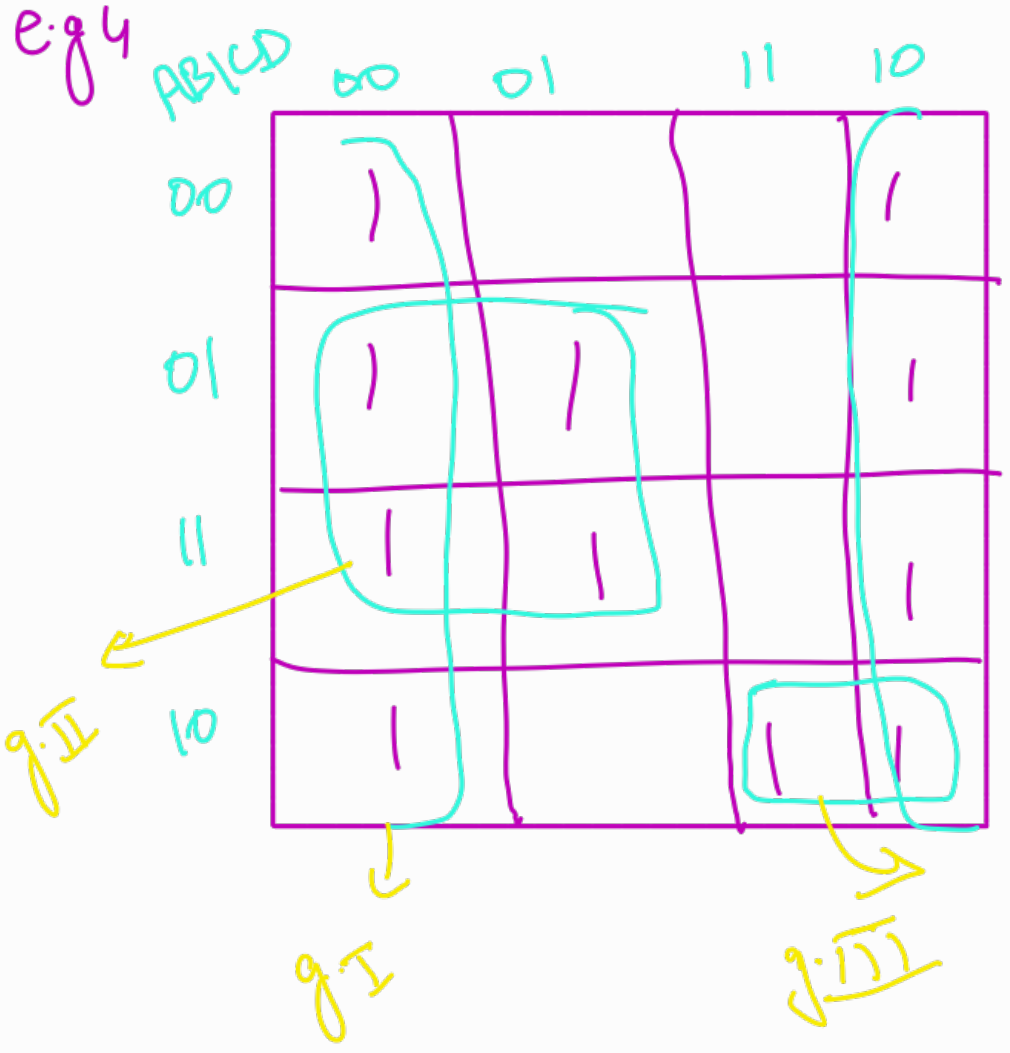
0 0

1 0

(x)

final Sop eq = $\bar{A} \cdot C + \bar{B} \cdot C + A \cdot \bar{C}$

e.g 4



Looking at g.I

C	D
0	0
1	0

Discard A and B

Discard C

$= \overline{D}$

g.II → A

A
0
1

B

B
1
1

→ $B \cdot \overline{C}$

C	D
0	0
0	1

(x)

g.III

A	B
1	0

C	D
1	1
1	0

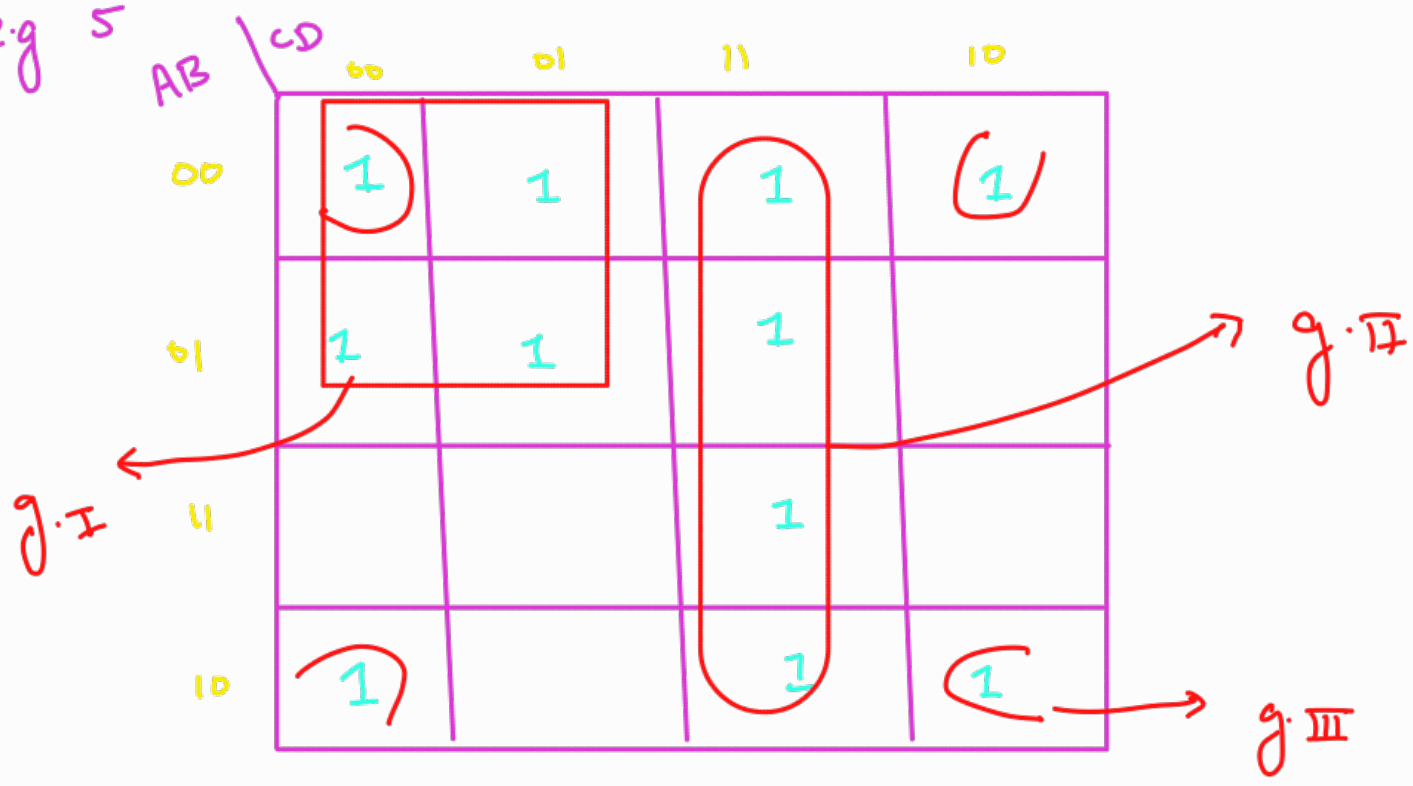
(x)

→ $A \cdot \overline{B} \cdot C$

Final Ans

$= \overline{D} + B \cdot \overline{C} + A \cdot \overline{B} \cdot C$

e.g 5



g.I

A	B
0	0
0	1
(x)	

C	D
0	0
0	1
(x)	

$\rightarrow \overline{A} \cdot \overline{C}$

g.II

A	B
(x)	(x)

C	D
1	1

$\rightarrow C \cdot D$

g.III

A	B
0	0
1	0
(x)	

C	D
0	0
1	0
(x)	

$\rightarrow \overline{B} \cdot \overline{D}$

final $\overline{A} \cdot \overline{C} + C \cdot D + \overline{B} \cdot \overline{D}$

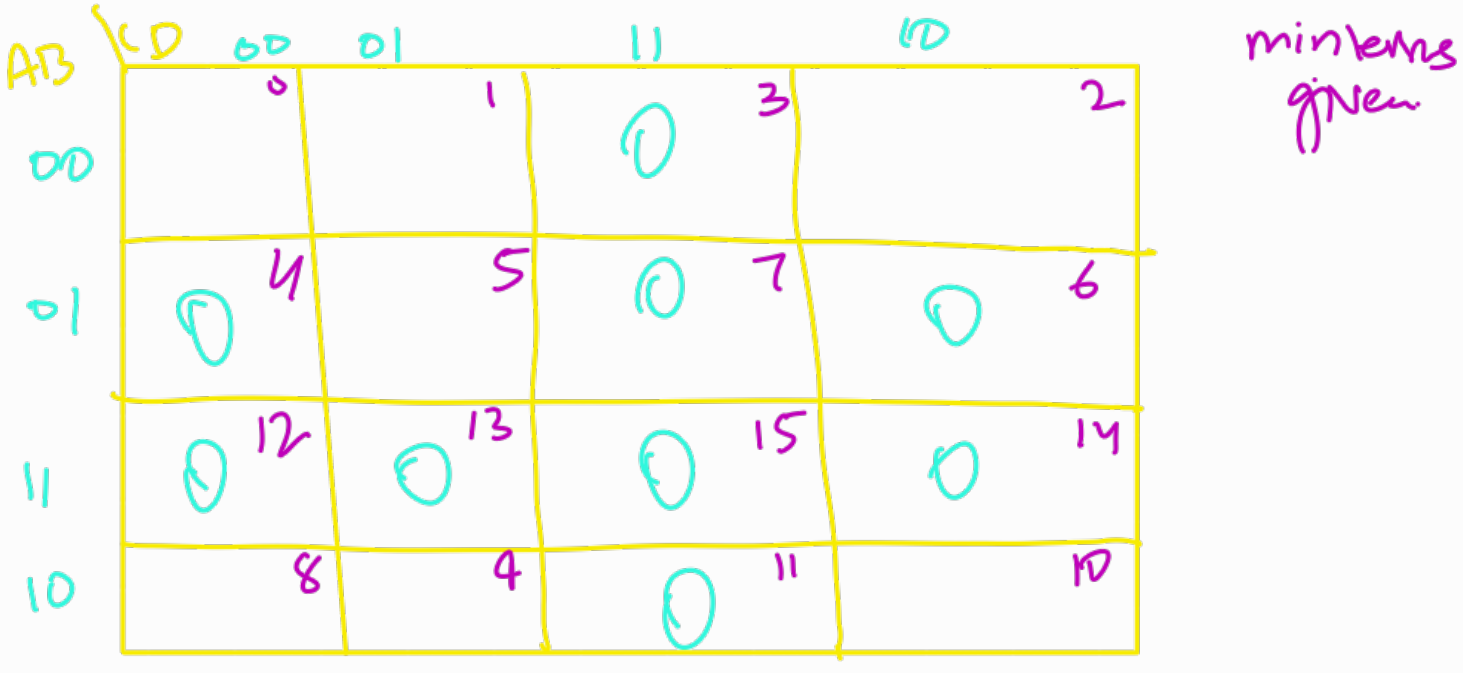
(Pos through k-maps)

0 → X normal variable

1 → $\overline{X}$ complemented variable

e.g $F = \sum m(0, 1, 2, 5, 6, 9, 10)$

Plot the k-map.

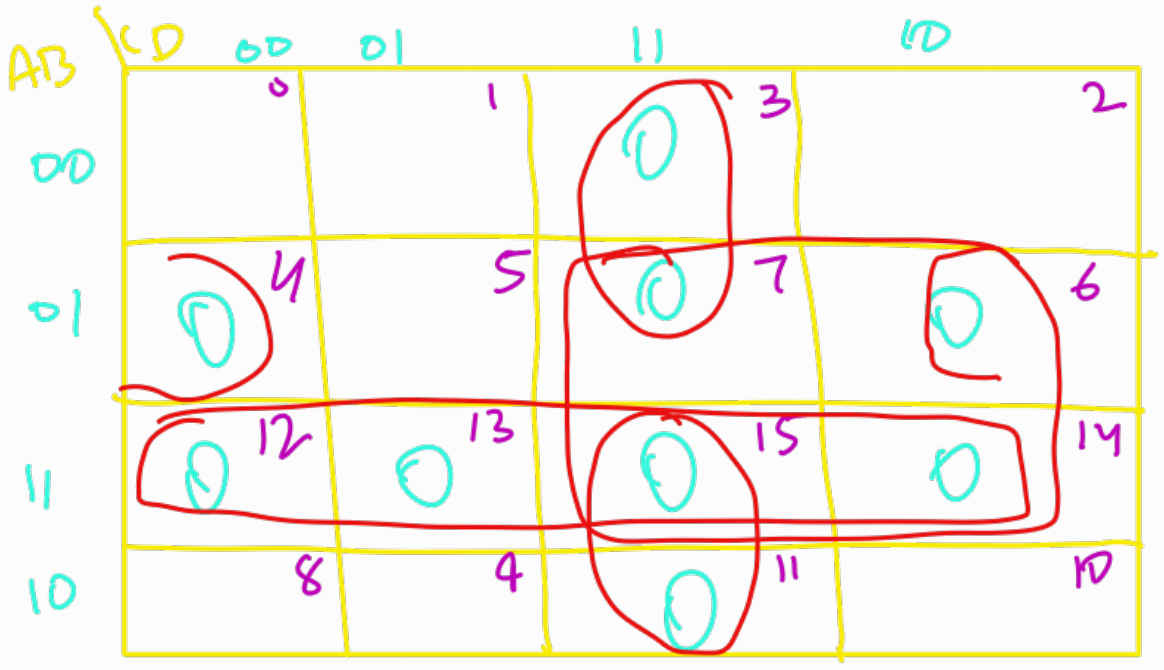


minterms given

finding pos

$F = \prod M(3, 4, 6, 7, 11, 12, 13, 14, 15)$

→ Maxterms plot 0 in T.T | k-map.



Same Rules for zeros groping.

for POS use

$$(\bar{C}+\bar{D})\cdot(A+B)\cdot(\bar{B}+\bar{C})$$

Don't care conditions "X"

→ output values are unspecified | we don't care about it. It can be minterm | maxterm we don't care

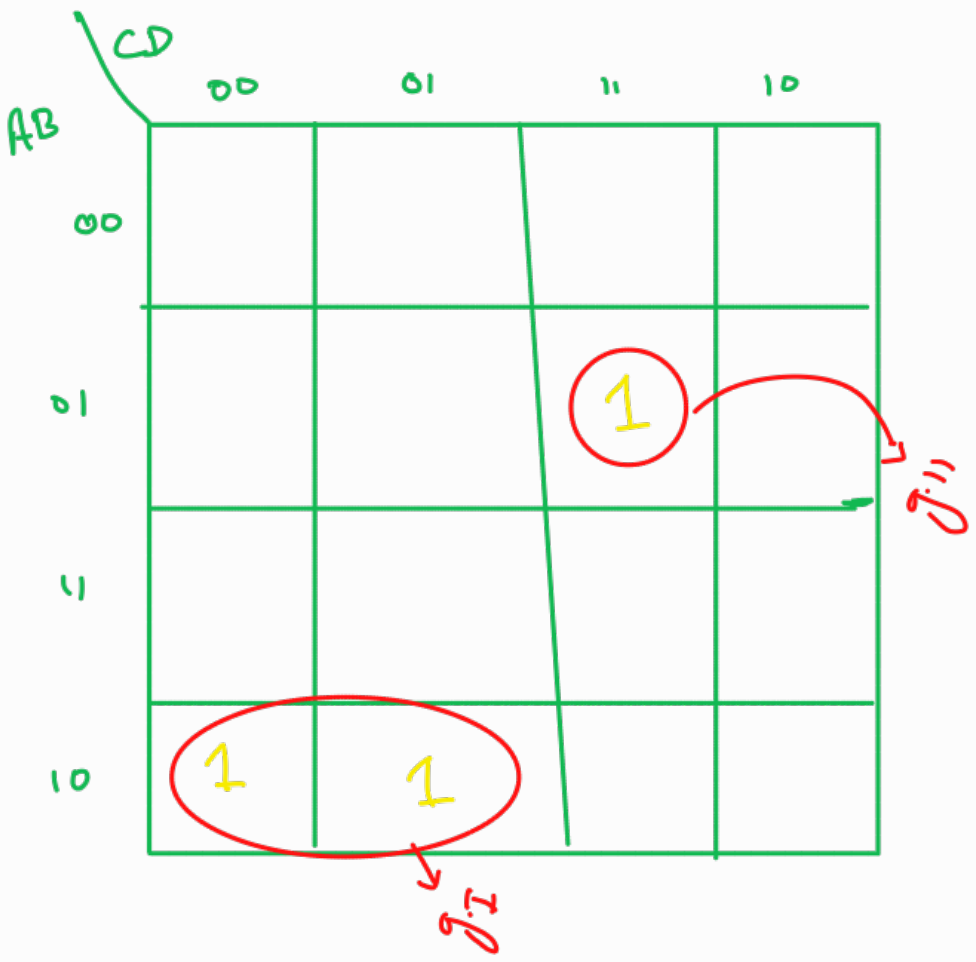
→ X can be assigned 0 or 1 it really doesn't matter as these terms will never occur.

→ Don't care terms are used to give advantage in making larger groups on K-maps

→ e.g of Don't care is BCD code values >=10.

→ It is not necessary that every X on K-map must be combined with every group.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X



Map Sop from T.T on K-map

A B
1 0

$A \cdot \bar{B} \cdot \bar{C}$

C D
0 0
0 1
(X)

g.I

finae $A\bar{B}\bar{C} + \bar{A}BCD$

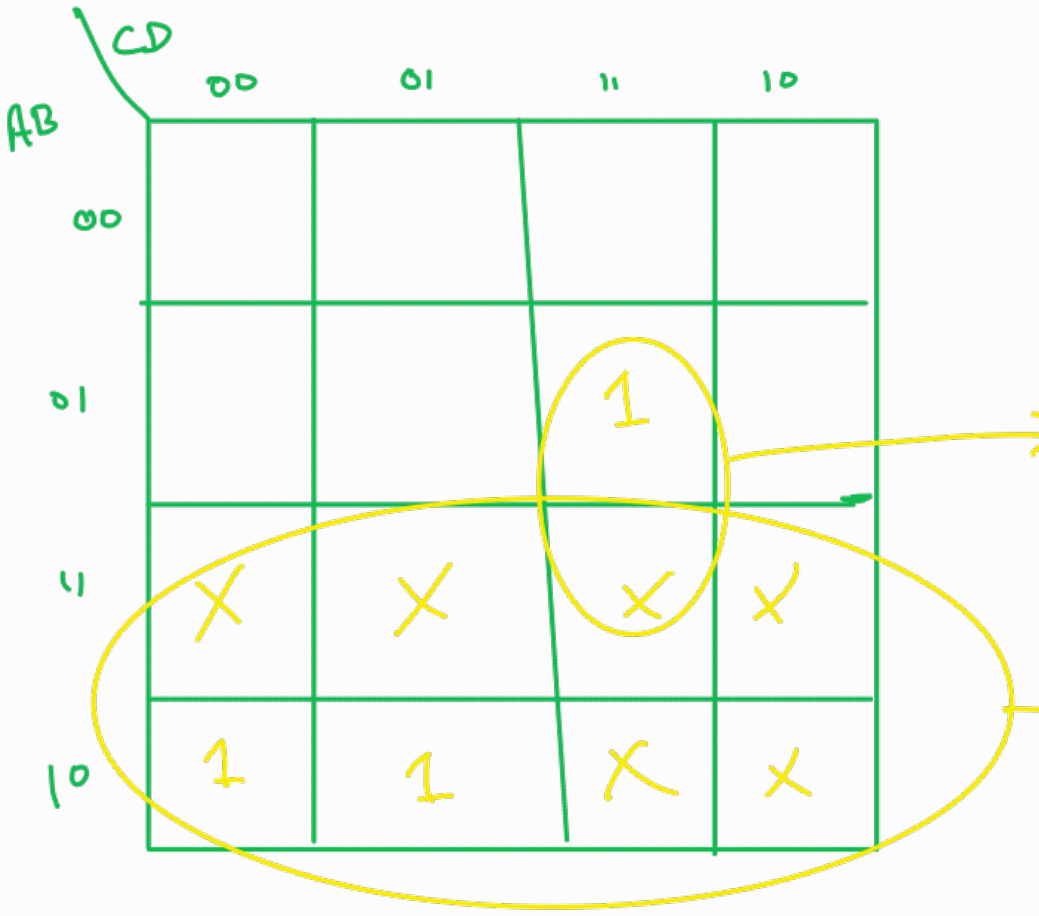
g.II

A B C D
0 1 1 1

$\bar{A} \cdot B \cdot C \cdot D$

now also mapping

Don't care



As we mapped X with 1's hence we assume there value is also 1.

g.I

A	B
1	1
1	0
	X

C D → Discard

→ A

g.II

A	B	C	D
0	1	1	1
1	1		
X			

→ B·C·D

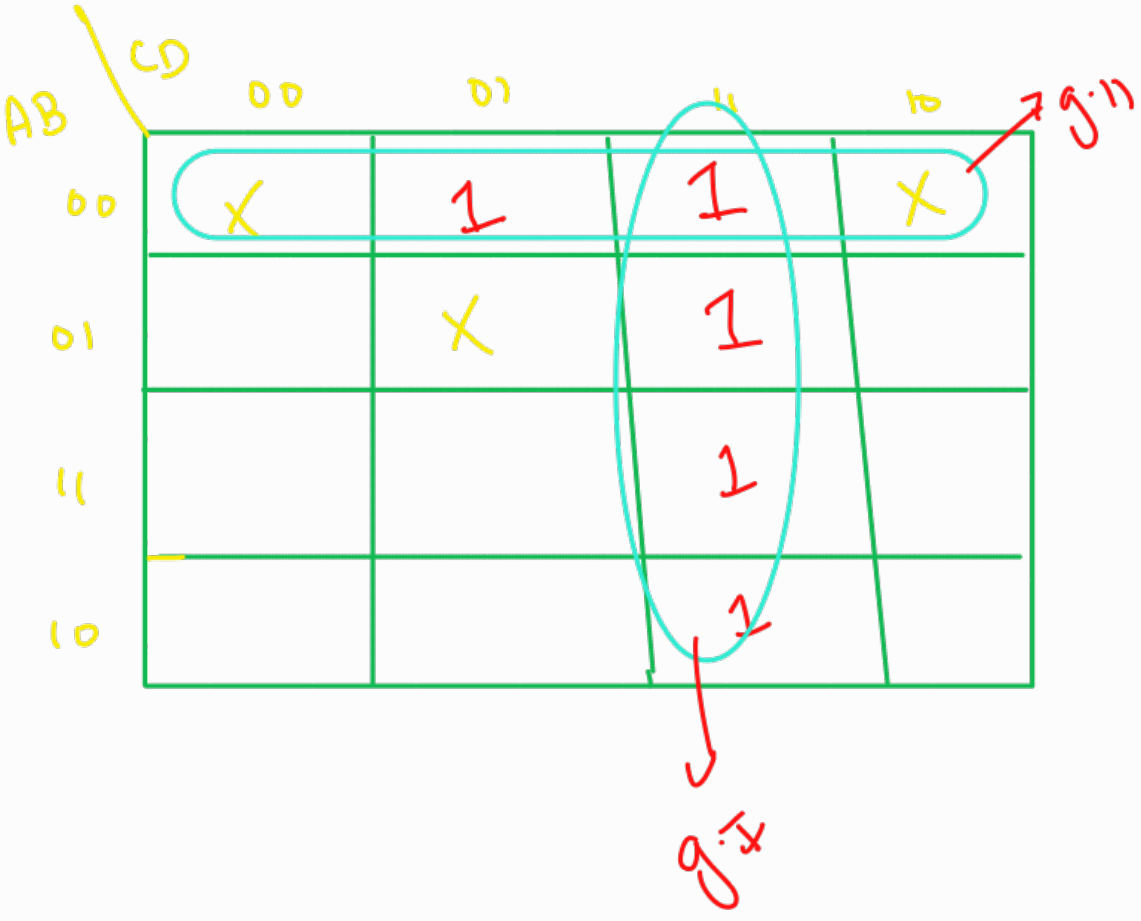
final **A + BCD**

hence using dont care it further minimzed our Sop expression by making Larger groups.

e.g Solve

$F = \Sigma m(1,3,7,11,15)$

$F = d(0,2,5)$ or $\phi(0,2,5)$ → these eq. means dont care



over-lapping of groups is allowed only if non-common 1/0 is present But not for X

Aside 0/1 on k-map X can be Left alone on k-map.

g.I

Discard A B

CD → 1 1

g.II

A B

0 0

CD discard

final $\boxed{C \cdot D + \bar{A} \cdot \bar{B}}$

Implicant → Every group of 1 is called Implicant

e.g 2^0, 2^1, 2^2 ....

Prime Implicant (P.I)

The largest possible group in k-map is called P.I

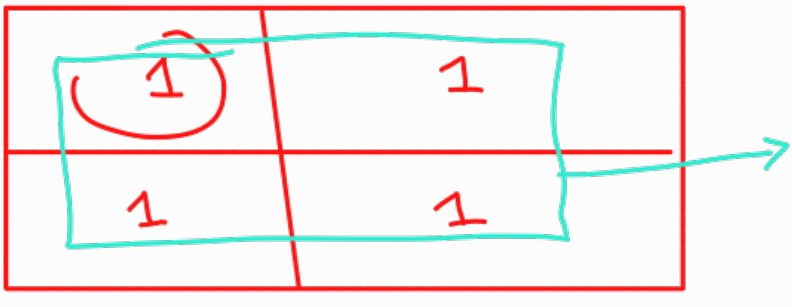
e.g 2

In case of 3var → Largest possible group is 8

In case of 4var → Largest possible group is 16
or any max no. of 1s.

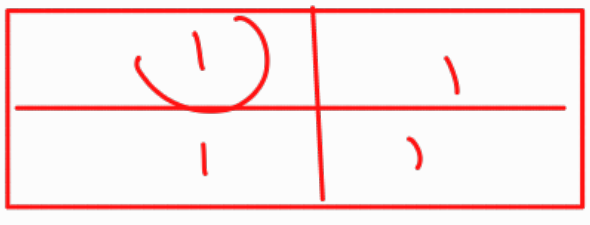
Essential P.I

The group of in which if we remove that group from k-map, the 1's in it become left alone. It can be a single leftalone 1 or more.



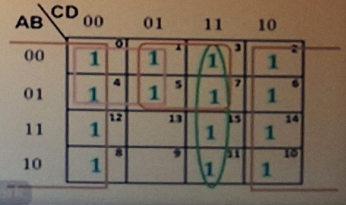
e.g if we remove this group.

3 1's will be leftalone which we cannot do in k-map.



POINTS TO REMEMBER FOR K-MAPS

1) Draw K-map
2) Enter function.
3) Identify Rectangle/groups
   - Look for **LARGEST** possible rectangle size, covering maximum number of 1s according to **'power of 2'** rule.
   - Each **"1 or 0"** (as per SOP/POS) on the map **must be** included in at least one a group.
   - The '1s' already in the group can be included in another group as long as over-lapping groups include the **non-common 1(s)** (i.e. the '1' which is left alone in K-map & not included in any group, we can combine it with already grouped 1s).
     - But, while covering this non-common 1(s), follow the rule of making largest possible group size.
4) Remove extra/redundant rectangles.
   - The rectangle which when removed has no effect on grouping (all 1s are still in groups)
   - **Do not** remove a rectangle so that any "1" after removal left un-grouped.
   - Remove extra rectangle/group after making & covering all possible groups.

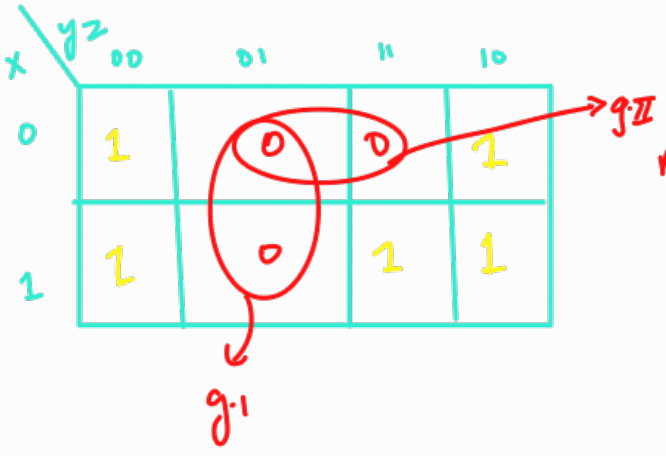


RECORDED WITH SCREENCAST MATIC
Protected with trial version of Visual Watermark. Full version doesn't put this mark.

Convert SOP to POS using K-maps

$F = x'y'z' + xy'z' + xyz' + x'yz' + xyz$

000 100 110 010 111



Now the remaining blocks mark them with 0 for POS.

g.I A discard B C

0 1

→ B+C'

g.II A=0 B C

0 1

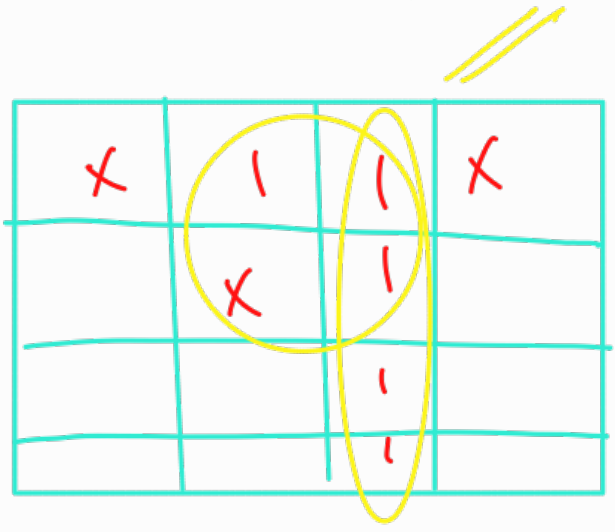
1 1

A+C'

(X)

final pos $\boxed{(B+C')(A+C')}$

note:: Make Sure to group least possible don't cares.



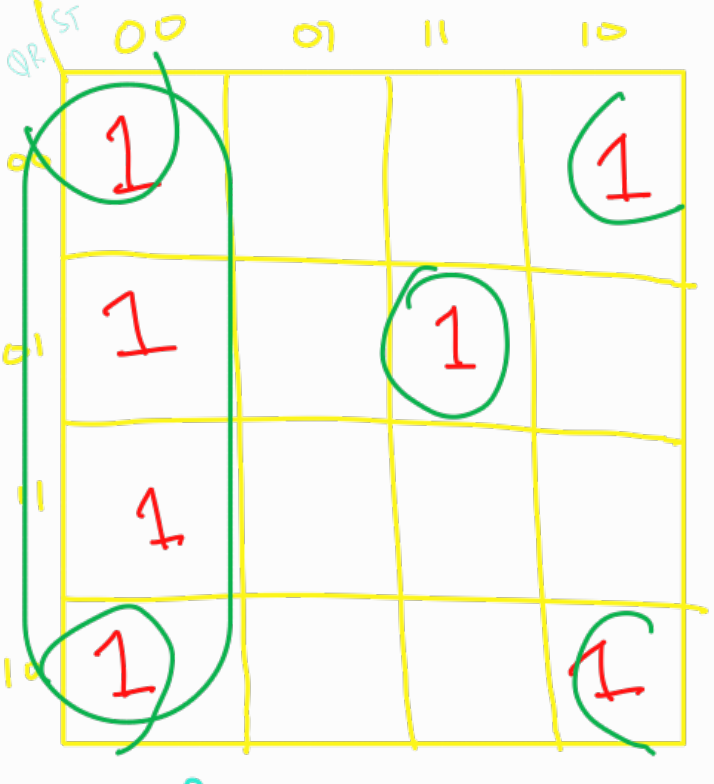
if X is left out no need to worry but make sure 1's are grouped

Property

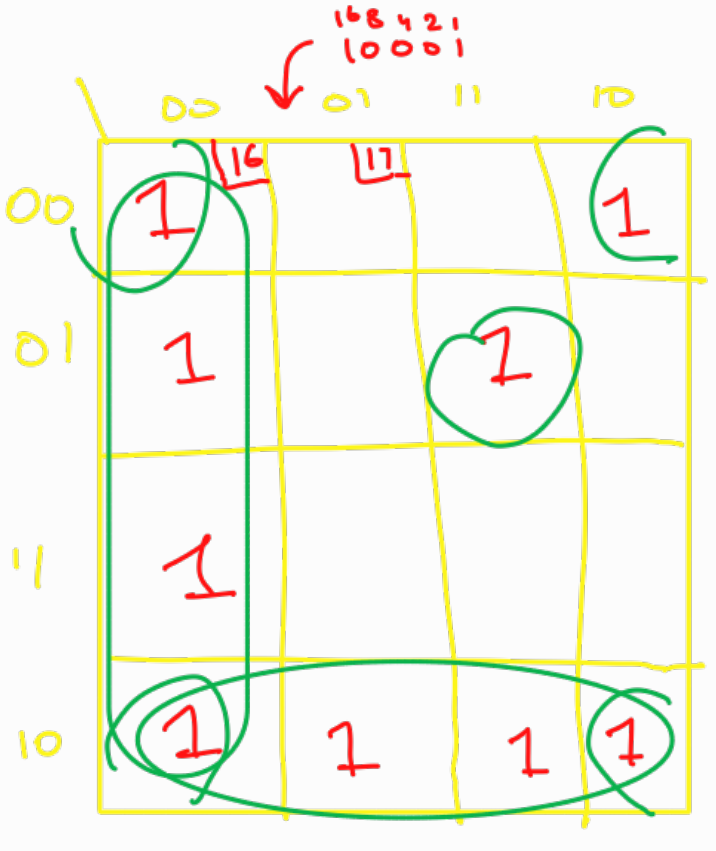
(5 variable K-maps)

$F(PQRST) = \Sigma(0, 2, 4, 7, 8, 10, 12, 16, 18, 20, 23, 24, 25, 26, 27, 28)$

$2^5 =$ 32 cells → making two 16 cells



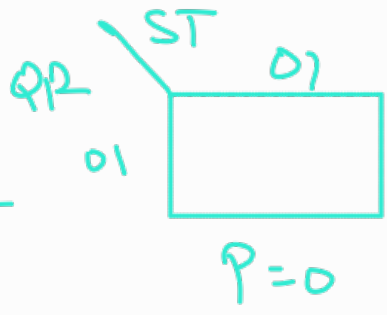
for this map P = 0



P = 1

lets say MSB is 0 for 1st k-map and 1 for Seloud k-map.

example naming of block



By using over-lapping

$P'Q'RS'T$

Universal gates → Nand and nor. For Simplest and fastest electronic result.

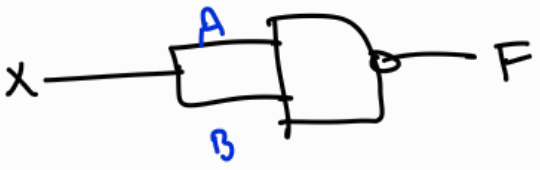
How to implement NOT gate using Nand only.



NAND has 2 inputs



Not has 1 input

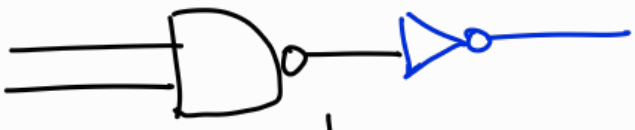


fulfils both Not and NAND inputs

X	A	B	F
0	0	0	1
0	0	0	1
0	0	0	1
0	0	0	1
1	1	1	0
1	1	1	0
1	1	-	0

Making AND gate using NAND





↓

hence we got AND gate.

OR gate



$x + y$

we know

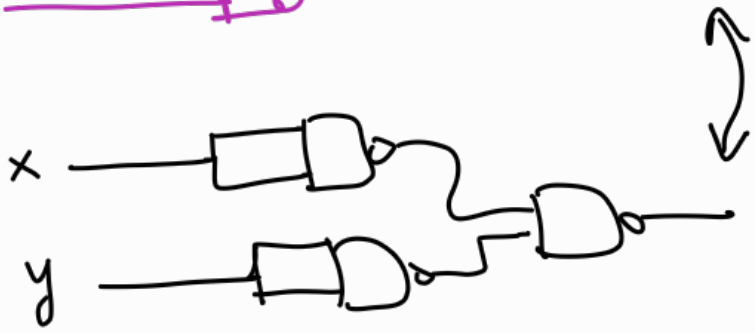
$\overline{F} = \overline{\overline{x+y}}$

$= \overline{\overline{x} \cdot \overline{y}}$



$\overline{x \cdot y}$



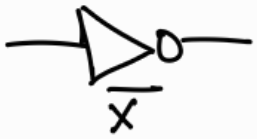


Same thing.

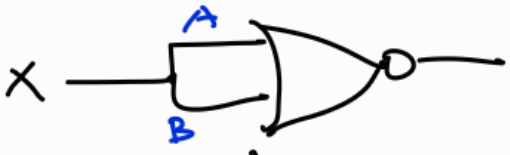
Now using NOR gate to implement all other gates.



$\overline{x+y}$



$\overline{x}$

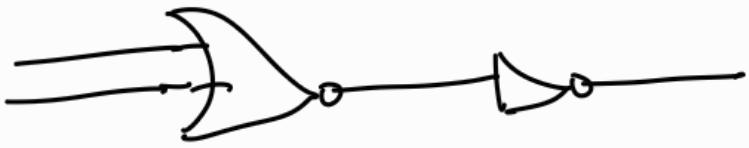


↓

Not gate

X	A	B	F
0	0	0	1
0	0	0	1
1	1	1	0
1	1	1	0

OR gate

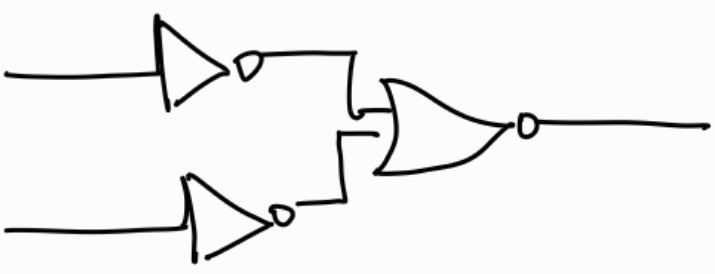


AND gate $= x \cdot y$ NOR $= \overline{x+y}$

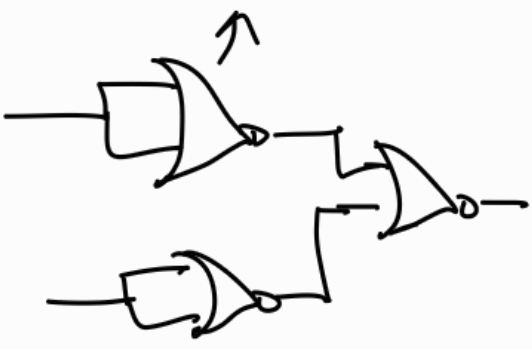
taking $F = \overline{\overline{x \cdot y}}$

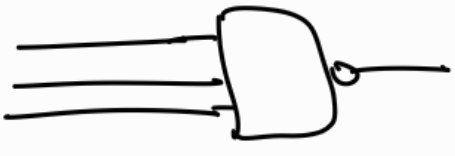
$F = \overline{\overline{x} + \overline{y}}$

3 NORs



OR





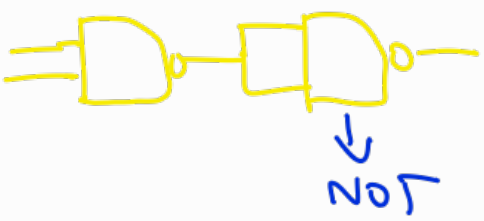
$\overline{A \cdot B \cdot C}$

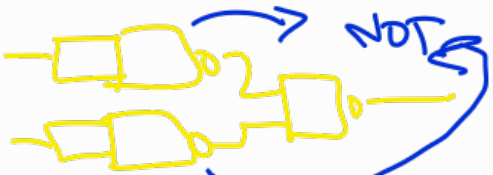


$\overline{A+B+C}$

Remember for nands.

NOT 

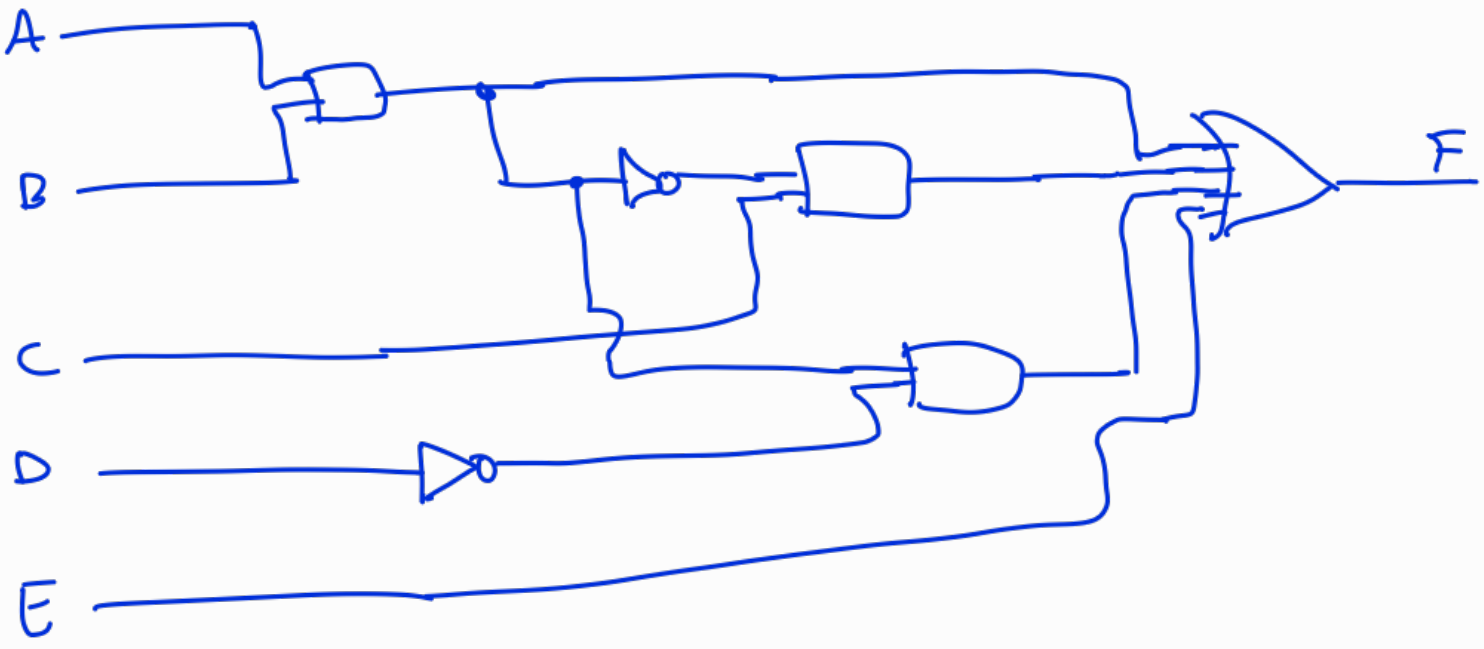
AND 

OR 

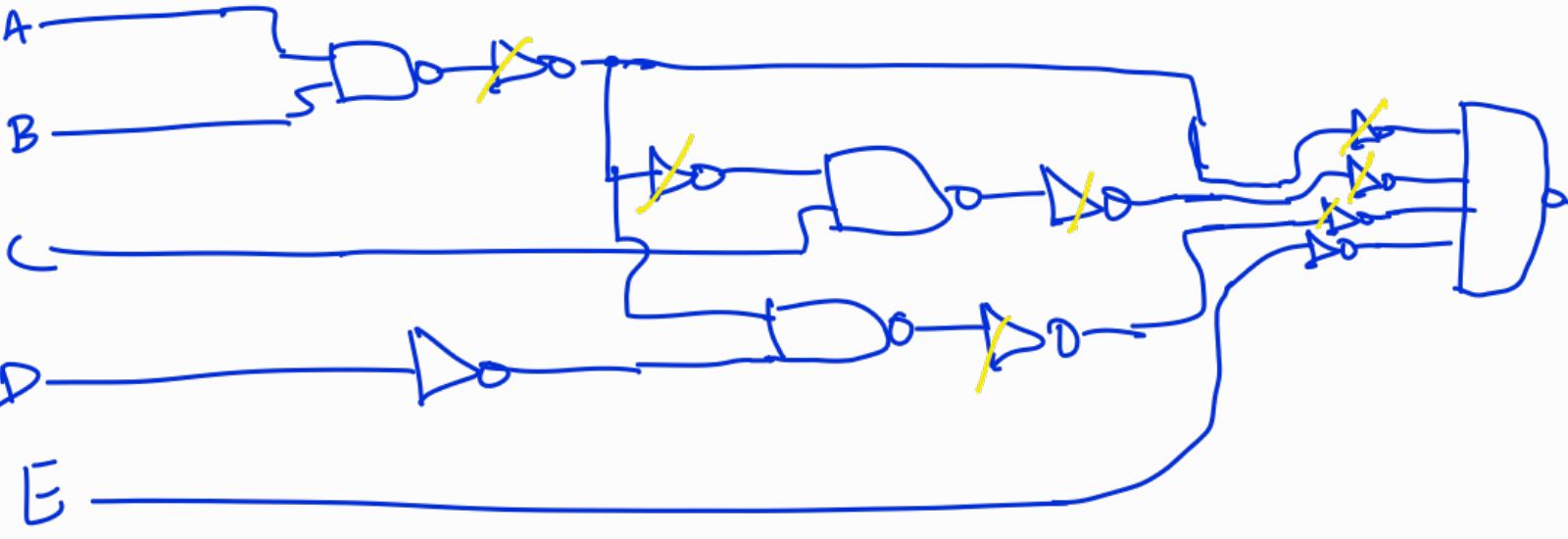
Implement $A \cdot B + (\overline{A \cdot B}) \cdot C + (\overline{A \cdot B}) \cdot D + E$

using NAND gates only.

First Draw the given eq.



Now replace all equalent logic gates with nand ones except NOT.



Cancel out the NOTS ⇒ double complement.

now replace the NOTS remaining with equivalent NANDS.

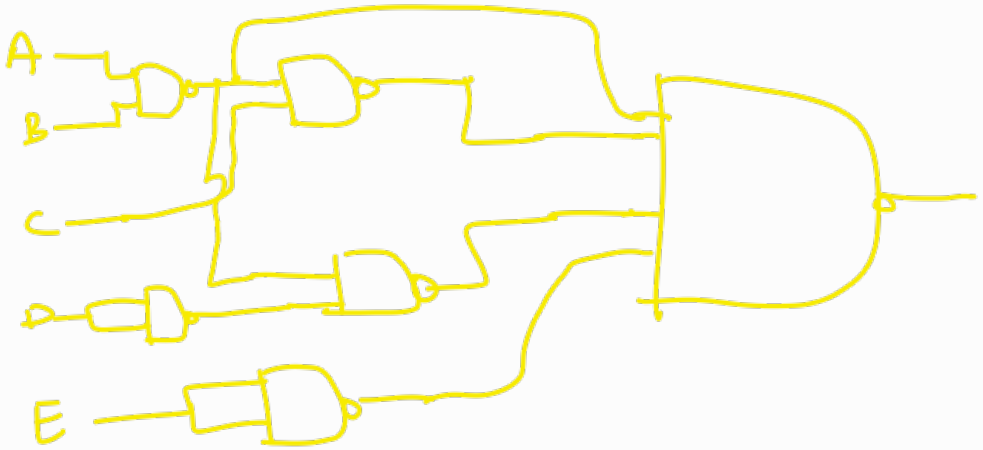
Another: Taking double complement.

$$F = A \cdot B + (\overline{A \cdot B}) \cdot C + (\overline{A \cdot B}) \cdot \overline{D} + E$$

$$\overline{F} = \overline{\overline{(A \cdot B) + (\overline{A \cdot B}) \cdot C + (\overline{A \cdot B}) \cdot \overline{D} + (E)}}$$

$$= (\overline{A \cdot B}) \cdot \overline{(\overline{A \cdot B}) \cdot C} \cdot \overline{(\overline{A \cdot B}) \overline{D}} \cdot (\overline{E})$$

$$= \overline{(\overline{AB}) \cdot [\overline{(\overline{AB})C}] \cdot [\overline{(\overline{A \cdot B})\overline{D}}] \cdot \overline{E}}$$



Implement using NOR gates only

$$F = A \cdot B + \overline{(A \cdot B)} \cdot C + \overline{(A \cdot B)} \cdot \overline{D} + E$$

$$F = \overline{\overline{(A \cdot B) + \left[\overline{(A \cdot B)} \cdot C\right] + \left[\overline{(A \cdot B)} \cdot \overline{D}\right] + E}}$$

$$F = \overline{\overline{(A \cdot B)} \cdot \left[\overline{\overline{(A \cdot B)} \cdot C}\right] \cdot \left[\overline{\overline{(A \cdot B)} \cdot \overline{D}}\right] \cdot \overline{E}}$$

$$F = \overline{(\overline{A} + \overline{B}) \cdot \left[\overline{(\overline{A} + \overline{B}) \cdot C}\right] \cdot \left[\overline{(\overline{A} + \overline{B}) \cdot \overline{D}}\right] \cdot \overline{E}}$$

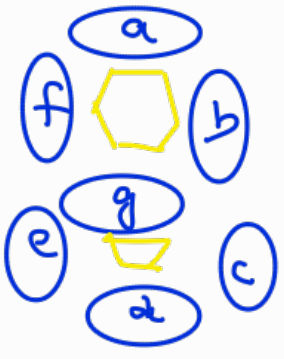
$$F = (\overline{A} + \overline{B}) \cdot \left[(\overline{\overline{A} + \overline{B}}) + \overline{C}\right] \cdot \left[(\overline{\overline{A} + \overline{B}}) + D\right] \cdot \overline{E}$$

$$F = \overline{(\overline{A} + \overline{B})} + \left[\overline{(\overline{A} + \overline{B}) + \overline{C}}\right] + \left[\overline{(\overline{\overline{A} + \overline{B}}) + D}\right] + E$$

(Seven Segment decoder)

Decimal digit in BCD ⟶ outputs Segments for the display.

Seven apples are a, b, c, d, e, f, g



Segments

0 – 9 digits can be displayed by these seven Segments

BCD → represented in 4 bits. A B C D. Hence 4 Inputs.

The Seven outputs a, b, c, d, e, f, g

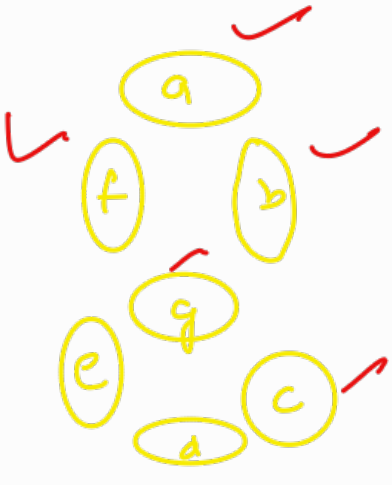
1 → on 0 → off

note digits from (10-15) 1010 to 1111 are Unnecessary and cannot be displayed.

Another option is to assign Zeros to Inputs from 1010 → 1111

Truth table

Inputs				outputs						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0							
0	0	1	1							
0	1	0	0							
0	1	0	1							
0	1	1	0							
0	1	1	1							
1	0	0	0							
1	0	0	1	1	1	1	1	1	0	1



To display → 9

